



Metaheuristic Optimization



Heuristic Algorithms in Transport and Finance

Metaheuristic Optimization

Prof. Dr. Angel A. Juan

Full Professor, Applied Stats and Operations Research Dept.

ajuanp@upv.es

ICSO
Analytics



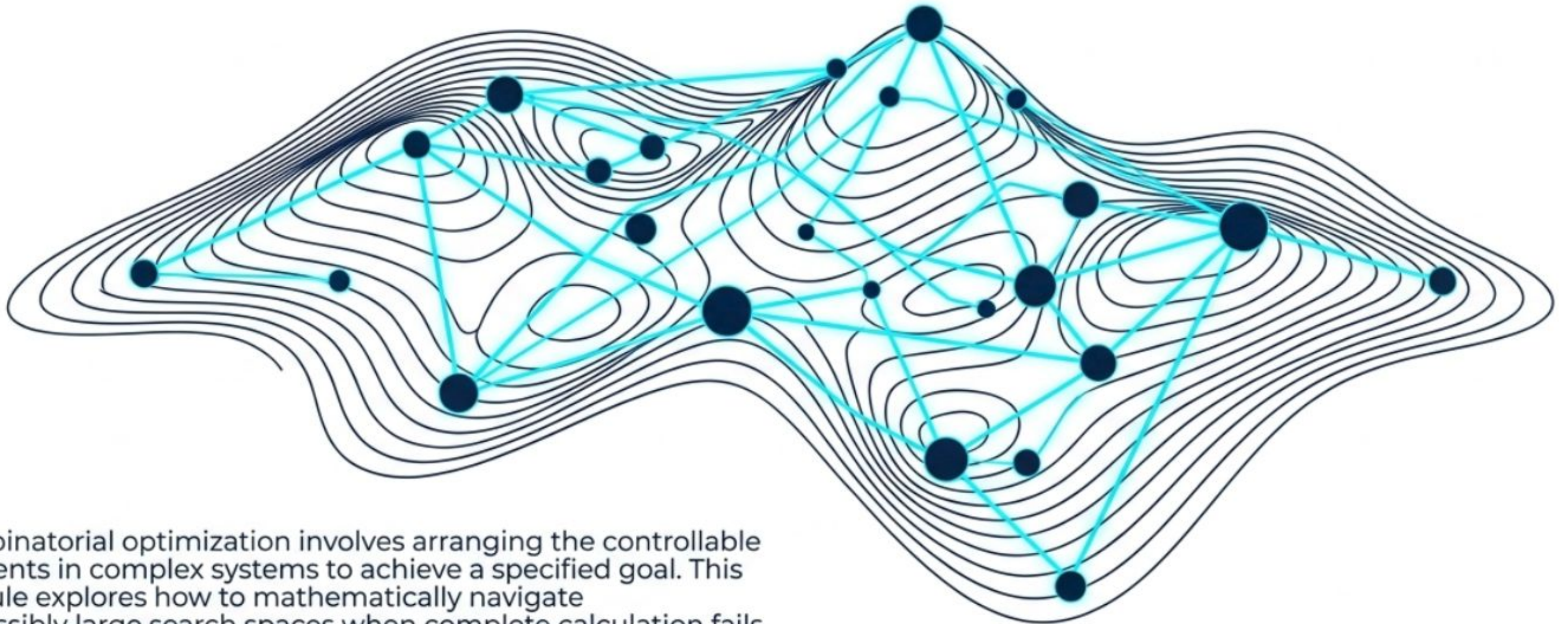
UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



valgrAI
Valencian Graduate School
and Research Network
of Artificial Intelligence

HEURISTIC AND METAHEURISTIC TECHNIQUES FOR OPTIMIZATION

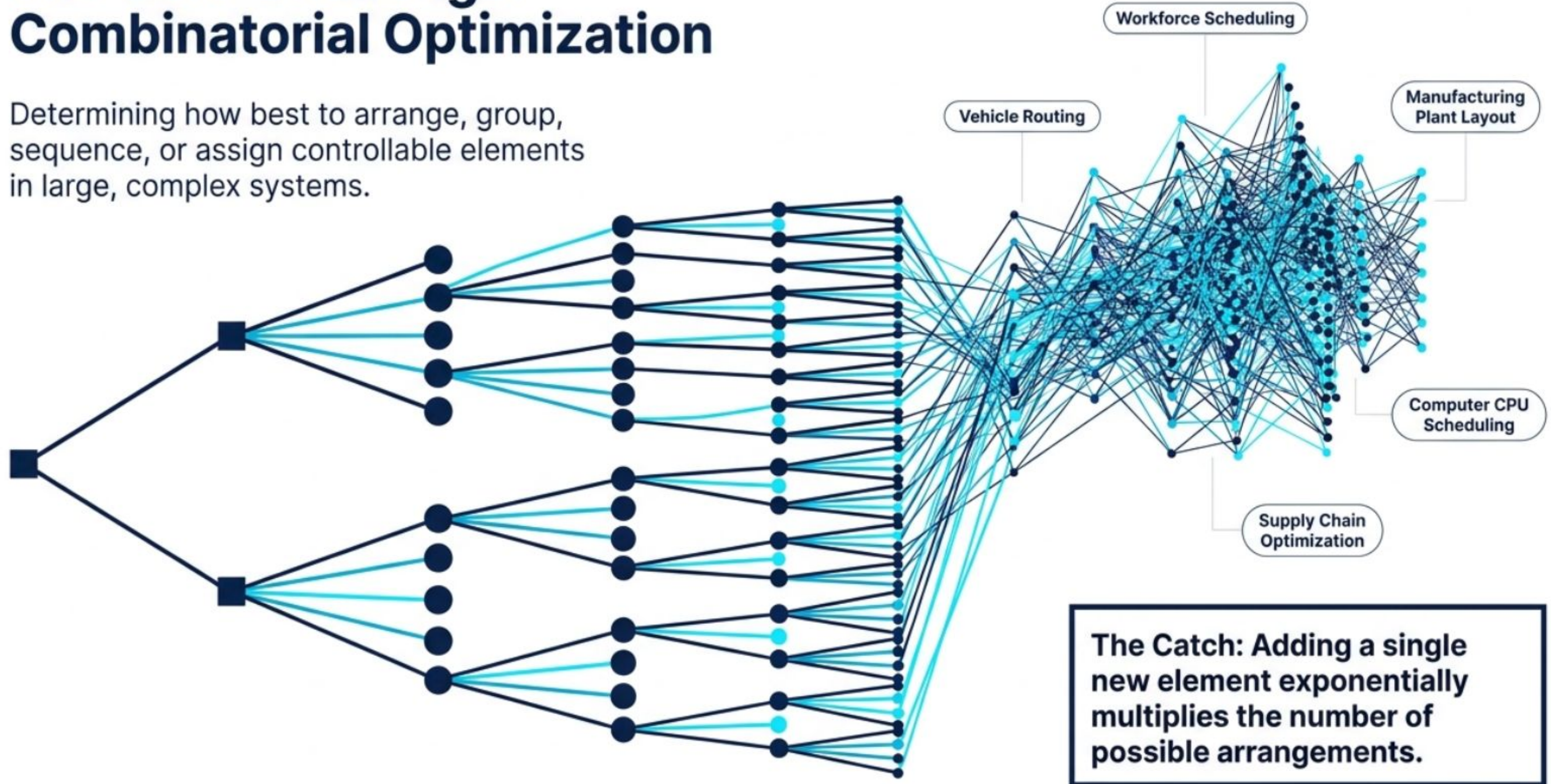
Navigating Combinatorial Complexity



Combinatorial optimization involves arranging the controllable elements in complex systems to achieve a specified goal. This module explores how to mathematically navigate impossibly large search spaces when complete calculation fails.

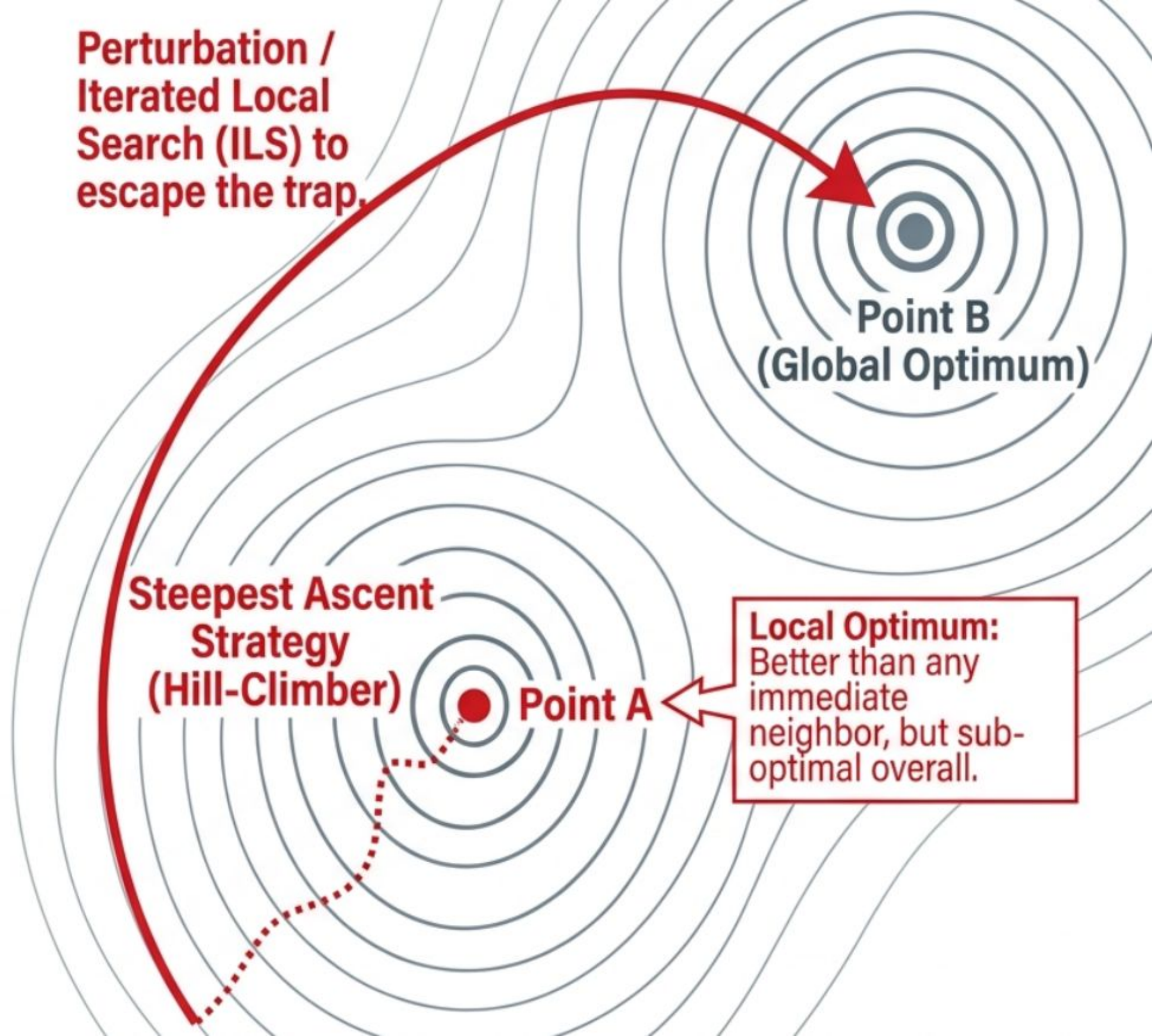
The Core Challenge: Combinatorial Optimization

Determining how best to arrange, group, sequence, or assign controllable elements in large, complex systems.



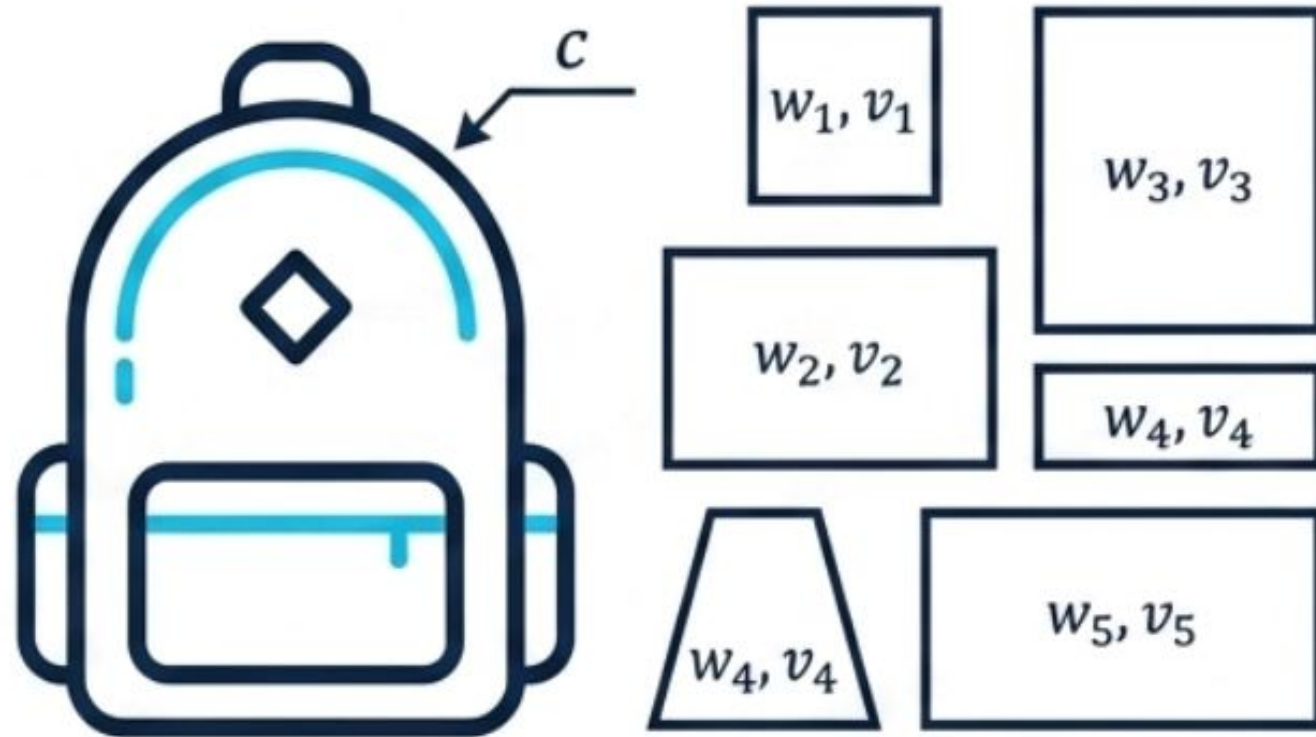
Combinatorial explosion makes proving exact optimal solutions impossibly slow for complex problems.

Metaheuristics provide strategic frameworks to find “good enough” solutions in a “small enough” computing time.



Famous Archetypes of Combinatorial Complexity

The Knapsack Problem

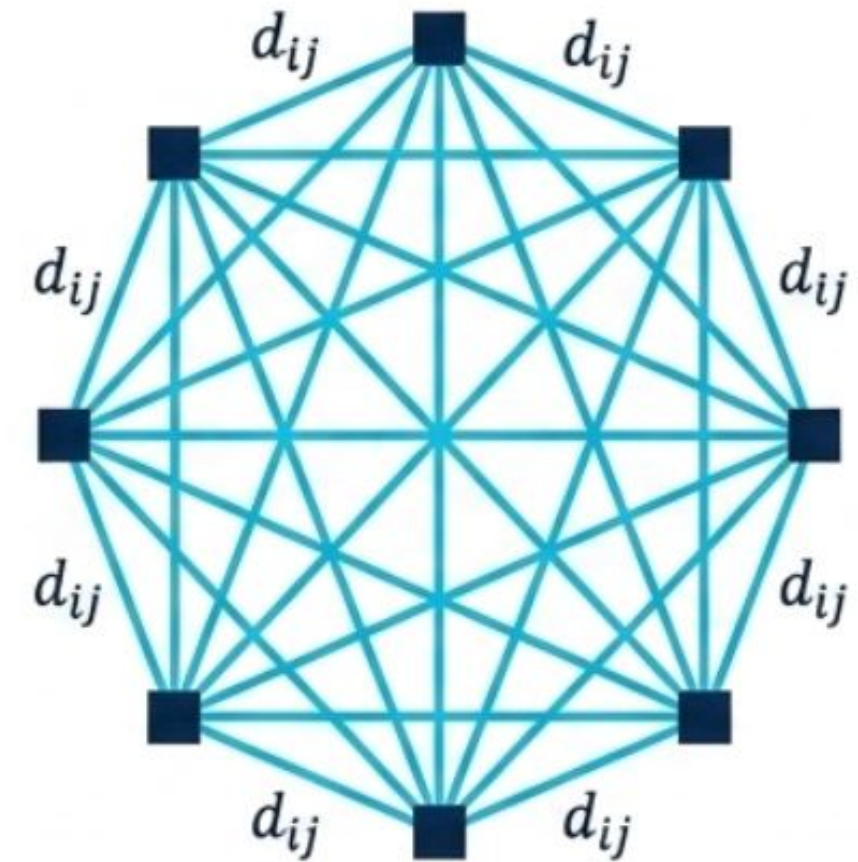


Decision: $x_i = 1$ (pack) or 0 (leave)

Objective: Maximize $\sum (v_i * x_i)$

Constraint: $\sum (w_i * x_i) \leq c$

The Traveling Salesman Problem (TSP)

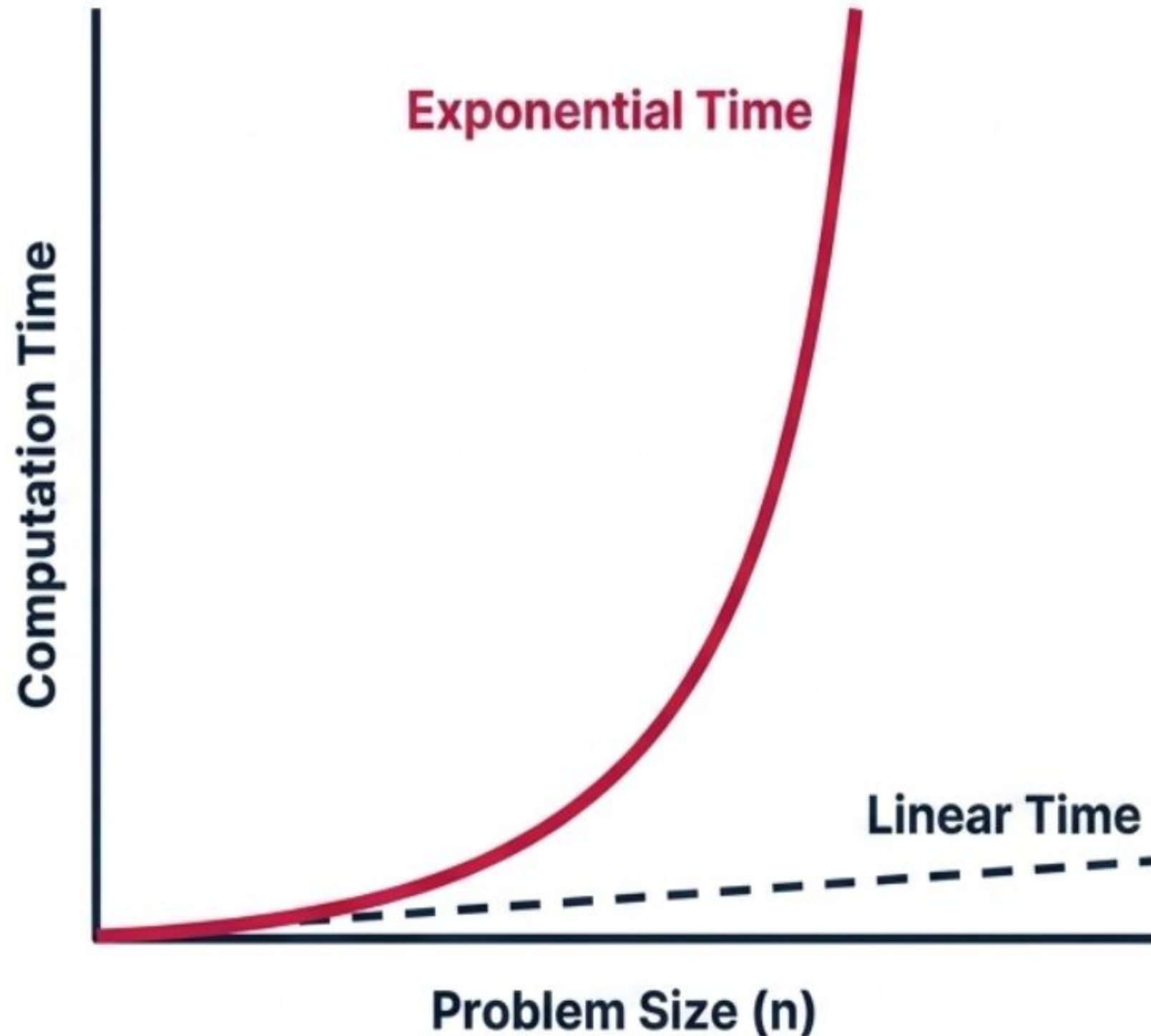


Decision: $x_{ij} = 1$ (travel i to j) or 0

Objective: Minimize total distance

Constraint: Enter and leave every city exactly once, excluding sub-tours.

The Computational Wall (NP-Hard)



The Complete Search Failure

For NP-hard problems (like Knapsack and TSP), finding a guaranteed optimal solution requires exhaustive search.

The Hardware Myth

Technological advances (faster CPUs) cannot overcome exponential time demands. A complete search of a moderately sized problem could take centuries.

The Necessary Pivot

We must abandon the search for perfect. We must instead engineer algorithms that search for near-optimal solutions efficiently.

The Optimization Spectrum

Exact Methods (MIP, Branch & Bound)	Heuristics	Metaheuristics (The Focus)
- Guarantee proof of optimality - Subject to exponential time (combinatorial explosion) - Require rigid linear problem formulation	- Fast, highly customized rules - Deeply problem-specific - Provide no guarantee of escaping local optima	- High-level, problem-independent frameworks - Flexible algorithmic design - Strategic mechanisms to escape local optima - Superior trade-off between quality and time

Metaheuristics do not demand objective functions be expressed as linear functions, offering unprecedented flexibility for real-life applications.

Three fundamental classes of metaheuristics are distinguished by how solutions are manipulated in computer memory.

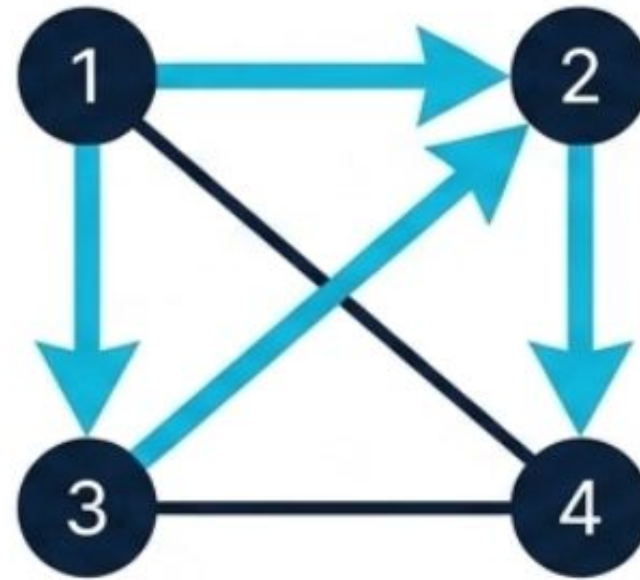


Level 1: Greedy Heuristics

Distance Matrix

	1	2	3	4
1		20	4	25
2	10		15	8
3	8	15		20
4	8	10	25	

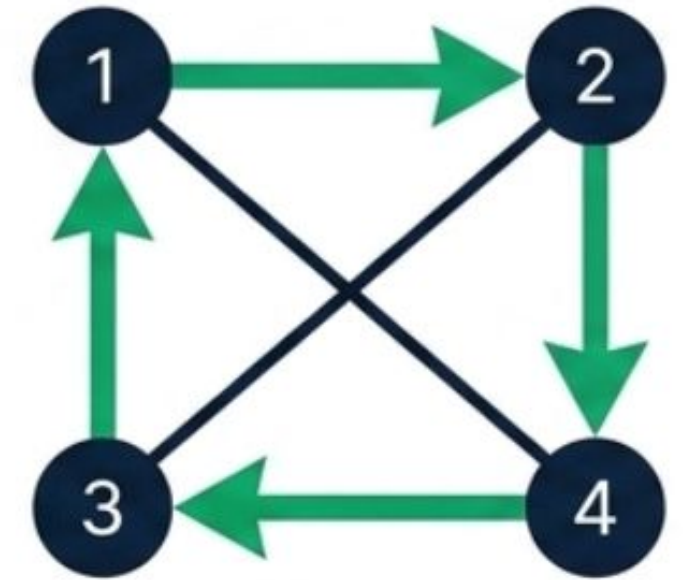
The Greedy Path



Total Distance = 52

Algorithm blindly picks the shortest immediate path.

The Optimal Path



Total Distance = 50

The actual best route.

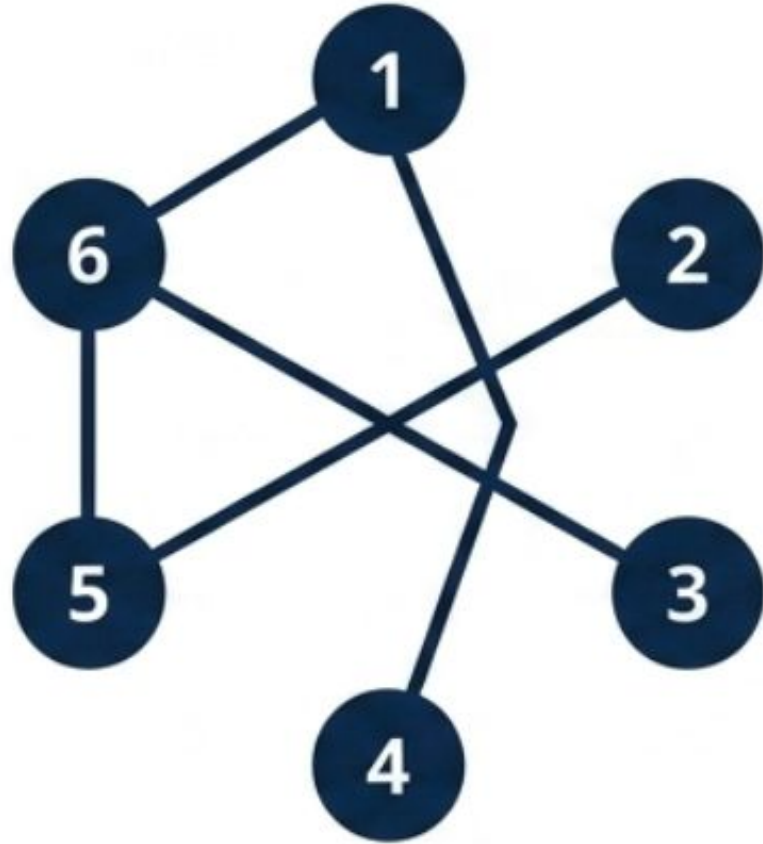
The Mechanic

Constructs a solution step-by-step based solely on immediate, short-term benefits (e.g., nearest unvisited city).

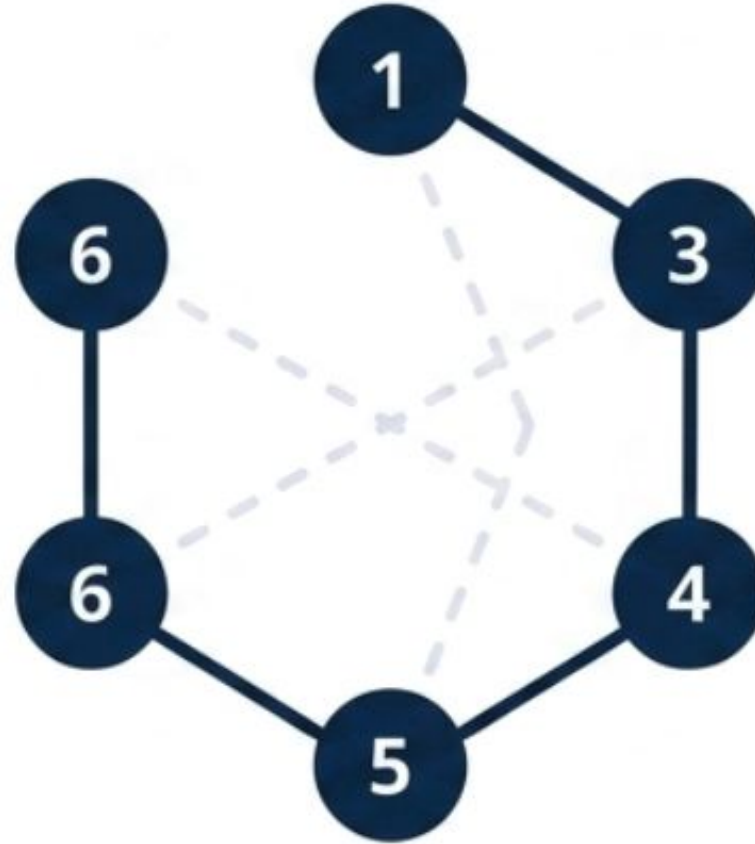
The Fatal Flaw

Greedy logic does not account for long-term consequences, practically guaranteeing a suboptimal total result.

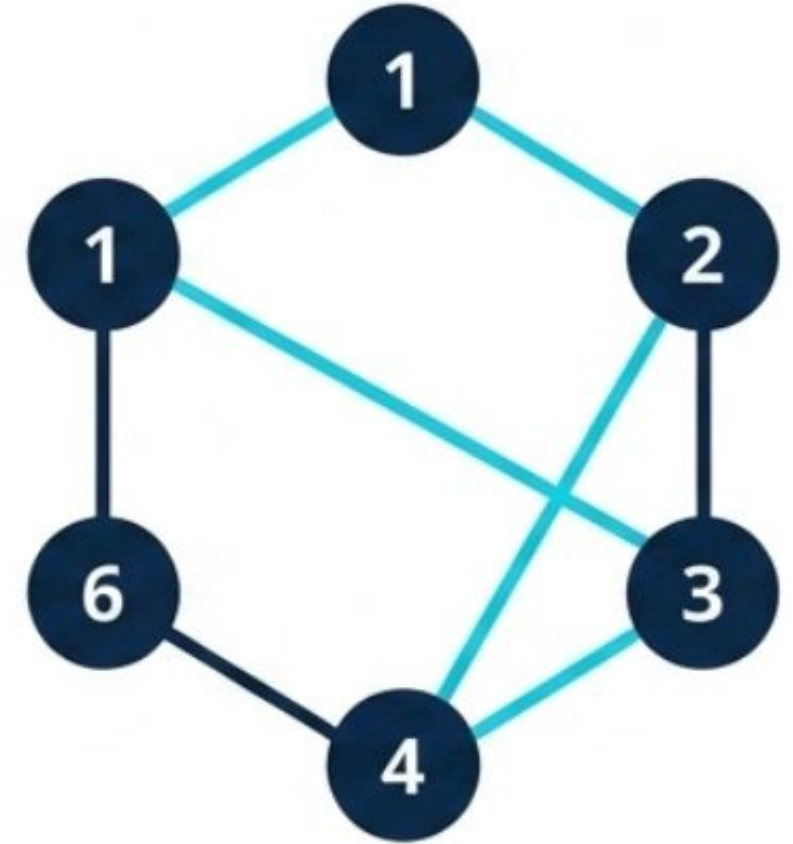
Level 2: Local Improvement Heuristics



Tangled Initial Route



Edge Removal (2-opt)



Improved Clean Route

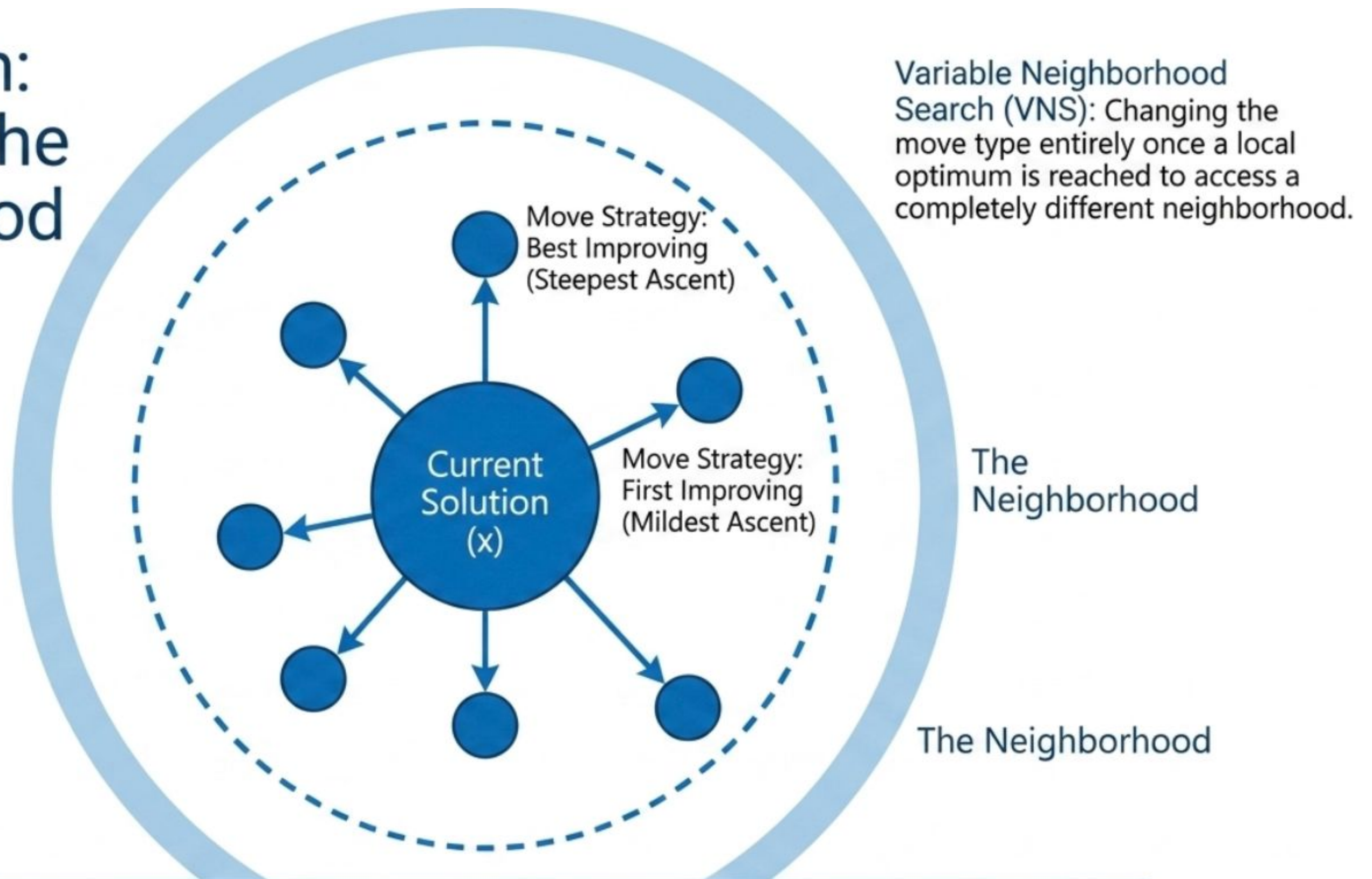
The Mechanic

Begin with an arbitrary configuration. Repeatedly make minor local modifications (such as edge swapping or k-opt facility exchanges).

The Rule

Adopt the change only if the objective function improves. Continue this process until no further improvements can be found.

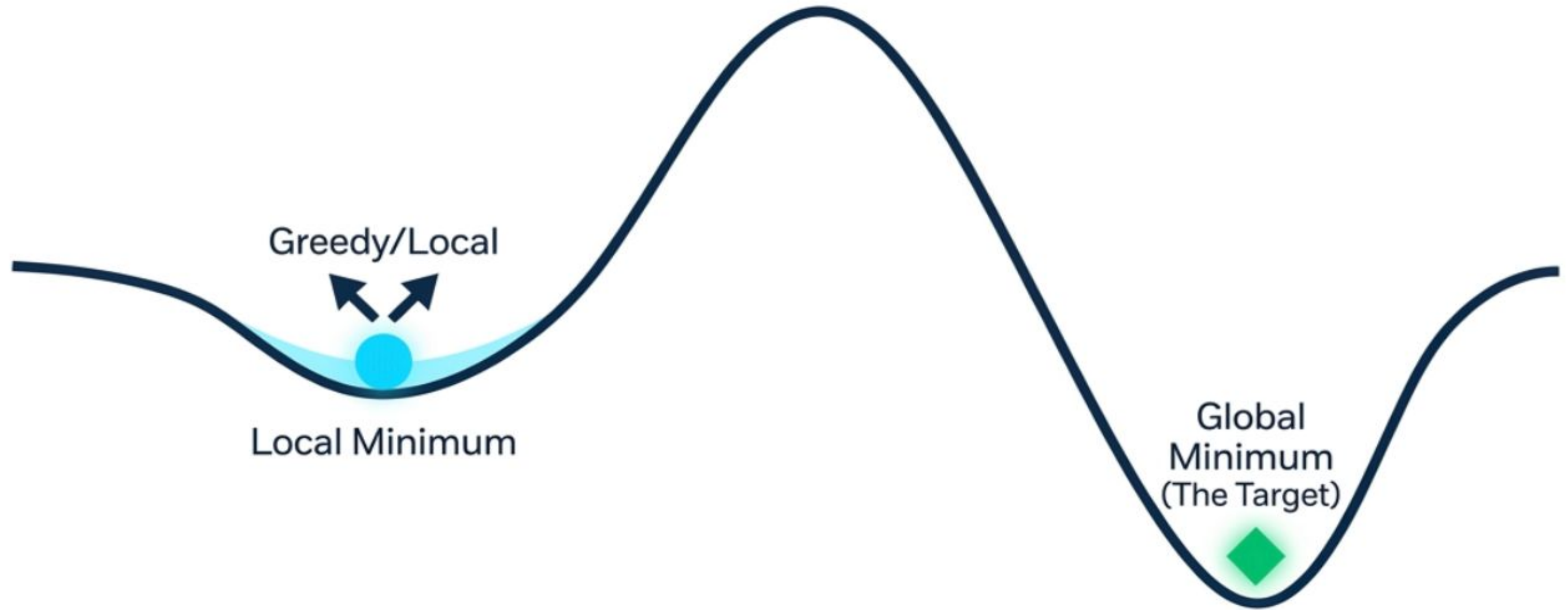
Local Search: Navigating the Neighborhood



Variable Neighborhood Search (VNS): Changing the move type entirely once a local optimum is reached to access a completely different neighborhood.

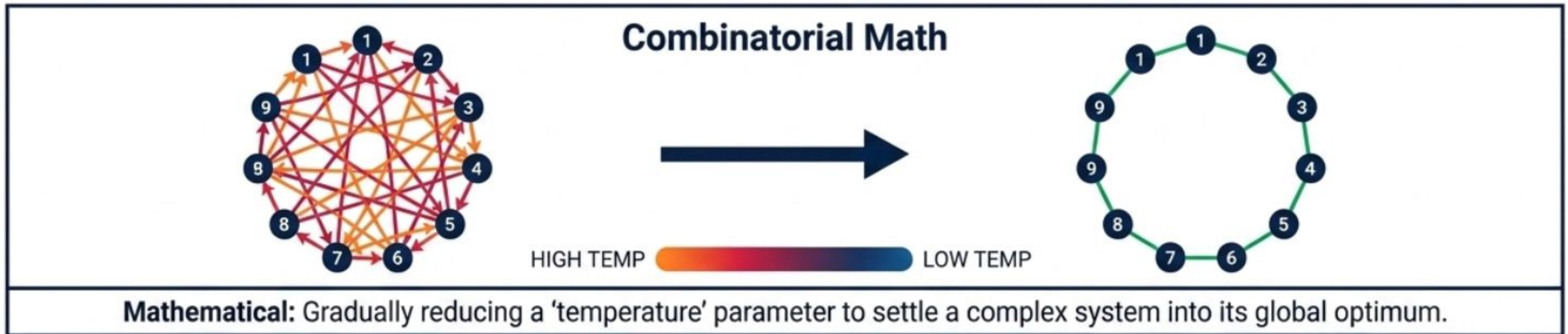
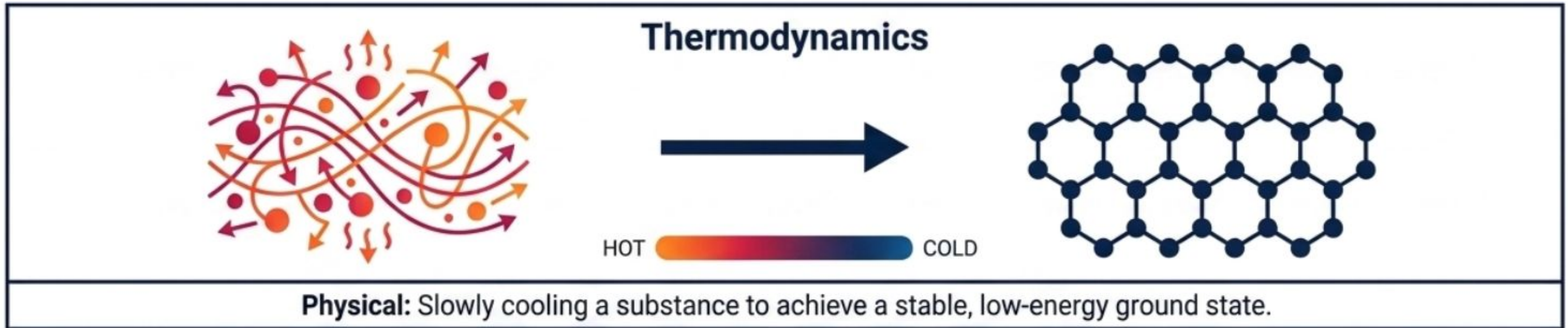
Local search navigates by making "small" adjacent moves. The specific rule used to select the next solution dictates the trajectory through the search space.

The Trap: Local vs. Global Optima



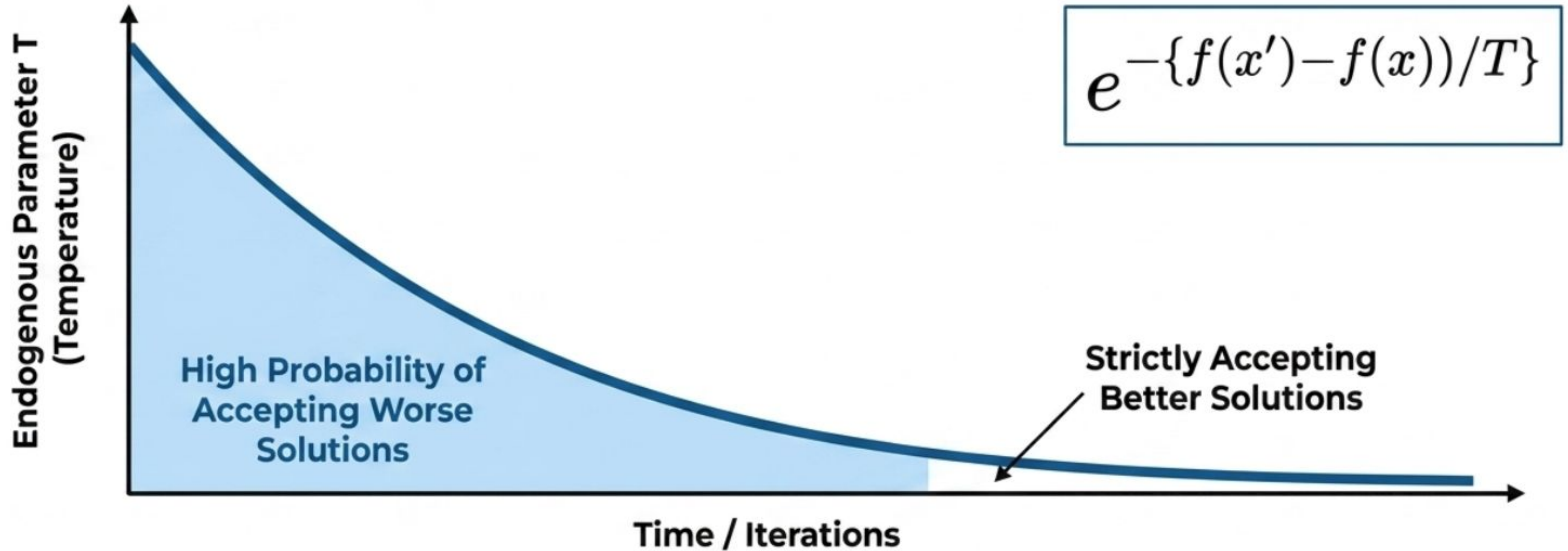
Deterministic algorithms move monotonically in an improving direction. Therefore, they are mathematically blind to the Global Minimum because reaching it requires temporarily accepting a worse solution to climb over the hill. We need a “Metaheuristic” to provide the algorithmic energy to escape this trap.

The Physics Breakthrough: Simulated Annealing



Developed in the 1950s for statistical mechanics; adapted for optimization in the 1980s. The thermodynamic analogy provides the foundation for escaping local optima.

Simulated Annealing: The Cooling Schedule



Mimicking the crystallization of solidifying metal, Simulated Annealing relies on a cooling schedule. It **intentionally accepts sub-optimal moves early in the search** to jump out of local optima, gradually freezing into a **strict hill-climber** as temperature drops.

The Engine of Escape: The Boltzmann Probability



Objective Change

Measures how much worse the proposed solution is.

$$P = e^{-\Delta F / \theta}$$



Temperature

Controls the algorithm's willingness to accept chaos.

The Magic Rule

If a move improves the solution ($\Delta F \leq 0$), accept it unconditionally. If a move is worse ($\Delta F > 0$), accept it probabilistically based on the Boltzmann equation.

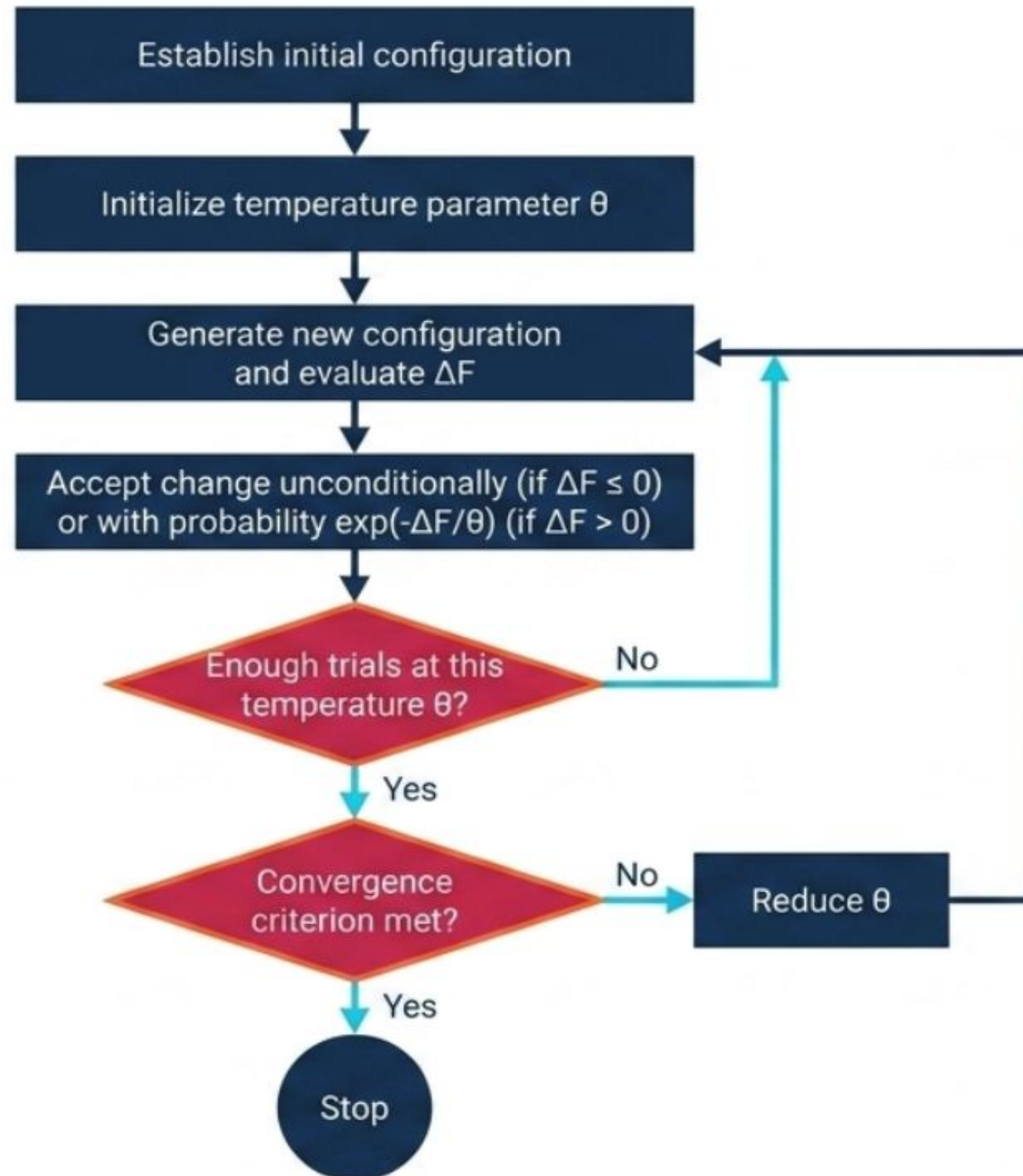
High Temperature

Early phase. The algorithm accepts many bad moves, creating a broad, coarse global search that bounces out of local traps.

Low Temperature

Late phase. The algorithm rejects almost all bad moves, behaving like a greedy local search to lock into the fine-tuned optimal state.

The Simulated Annealing Algorithm Flow



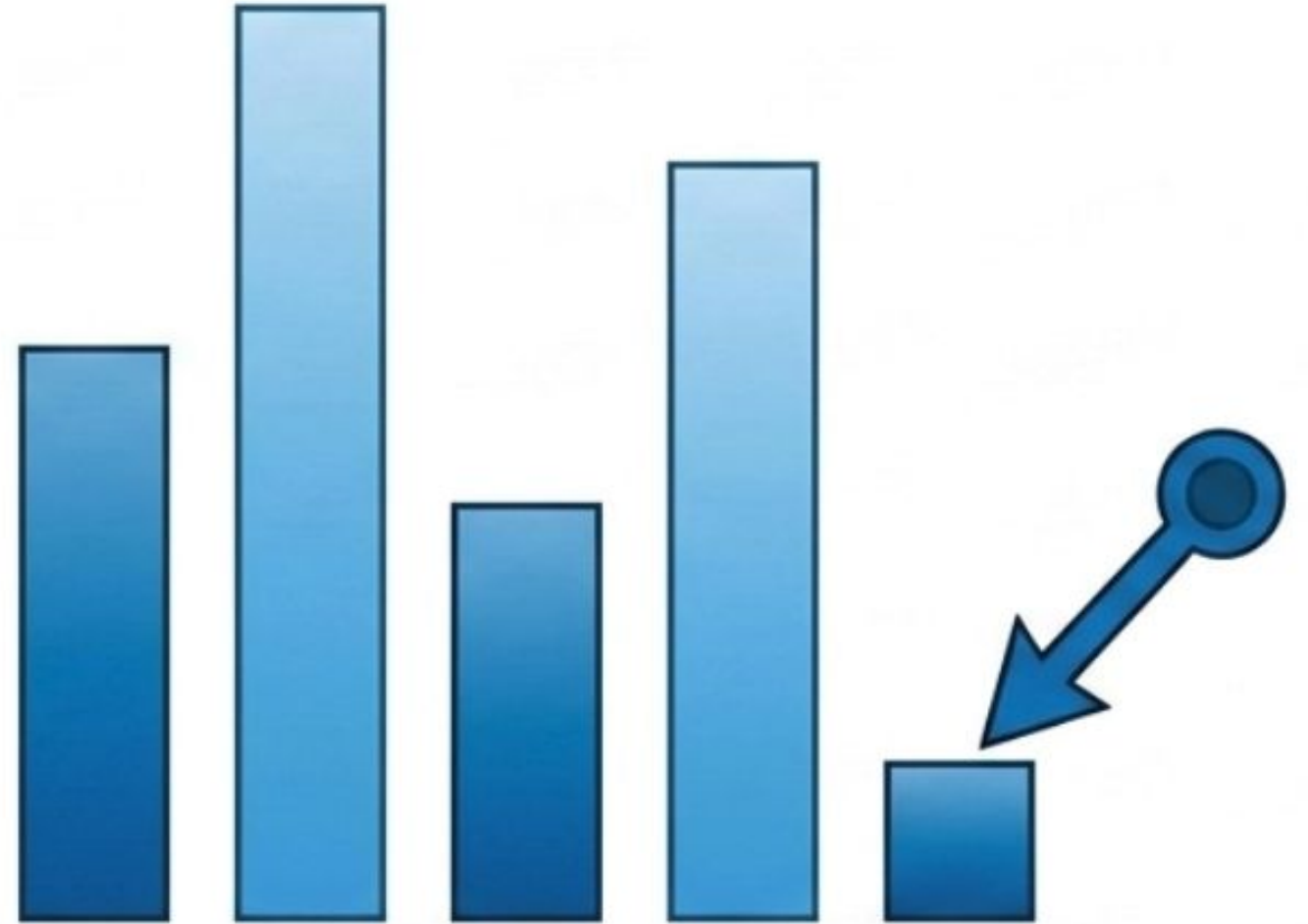
Synthesis Matrix: Navigating the Trade-Offs

Method	Execution Speed		Optimality Guarantee		Risk of Local Traps		Complexity
Complete/Exact Search	Infinitely Slow		 100% Perfect	 Zero Risk	 Low		
Greedy Heuristic	Lightning Fast		 Very Low	Guaranteed Trap 	 Low		
Local Search (k-opt)	 Fast	 Moderate	 High Risk	 Moderate			
Simulated Annealing	 Slower	 High (Near-Optimal)	 Escapes Traps	 High			

Tabu Search: Adaptive Memory Programming



Short-term Memory (Tabu List):
Forbids recently visited solutions to prevent infinite loops.



Long-term Memory (Frequency Memory): Guides search to seldom-visited regions or penalizes over-used elements (Guided Local Search).

Tabu search transforms optimization into an Adaptive Memory Programming framework, explicitly tracking the search trajectory to penalize repetition and force diversification.

Constructive Search: GRASP Framework

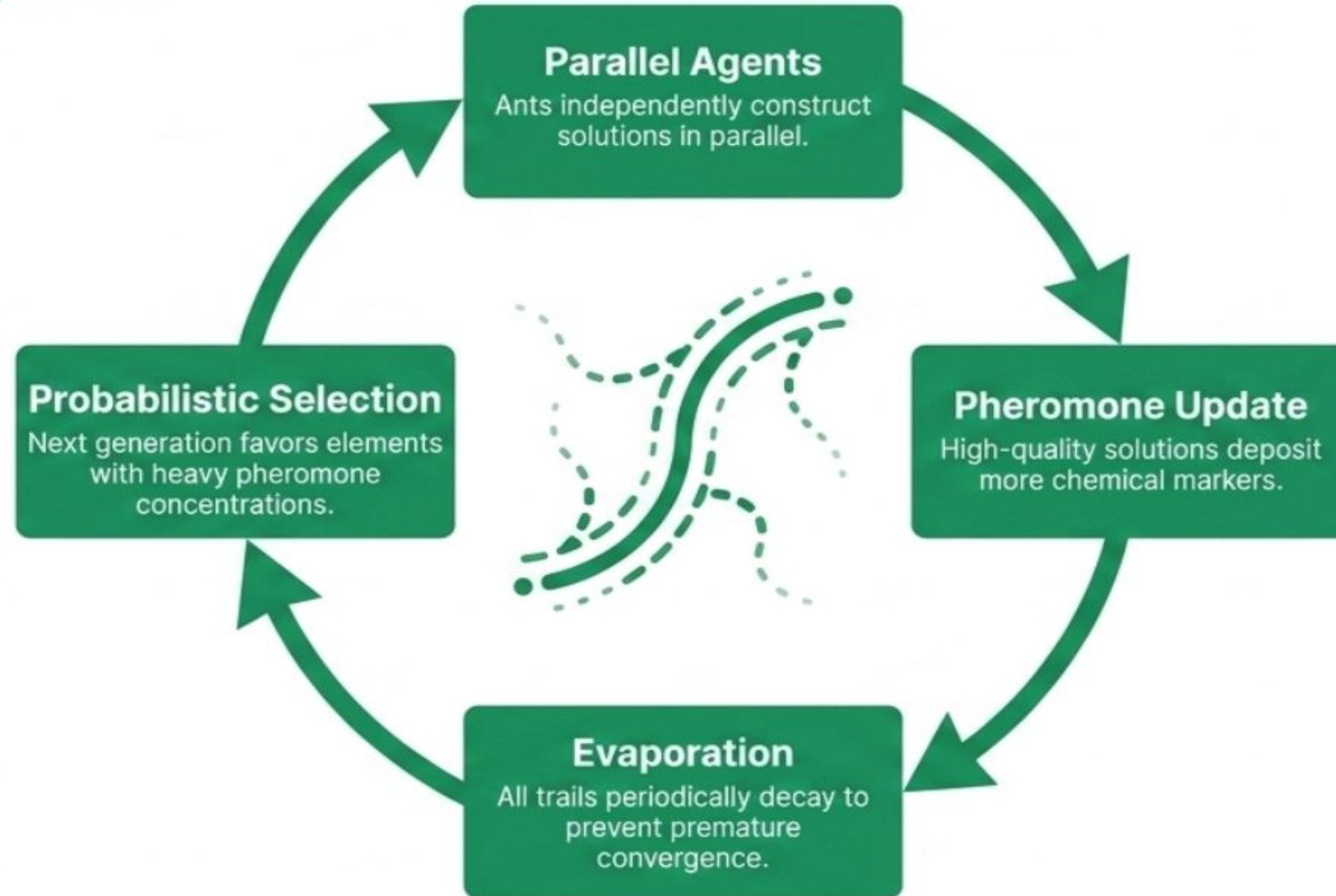


GRASP (Greedy Randomized Adaptive Search Procedure)



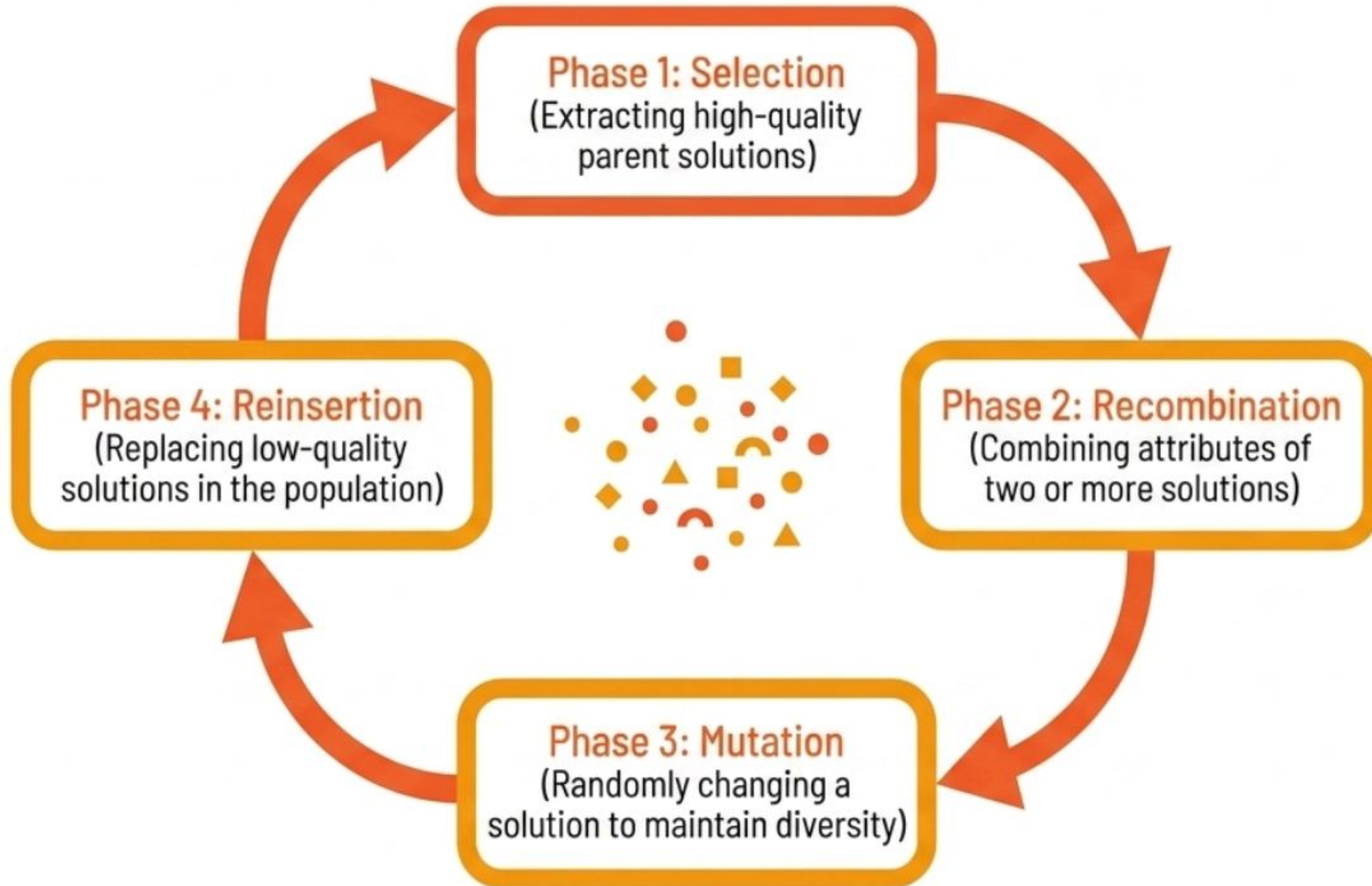
Constructive heuristics build from scratch. GRASP dampens the fatal flaw of pure greedy algorithms (short-sightedness) by injecting controlled randomness into the construction phase.

Ant Colony Optimization: Swarm Intelligence



ACO uses artificial pheromones as an external parameter. Elements belonging to better overall solutions receive higher chemical markers, subtly guiding parallel agents toward globally optimal combinations.

Population-Based: The Evolutionary Loop

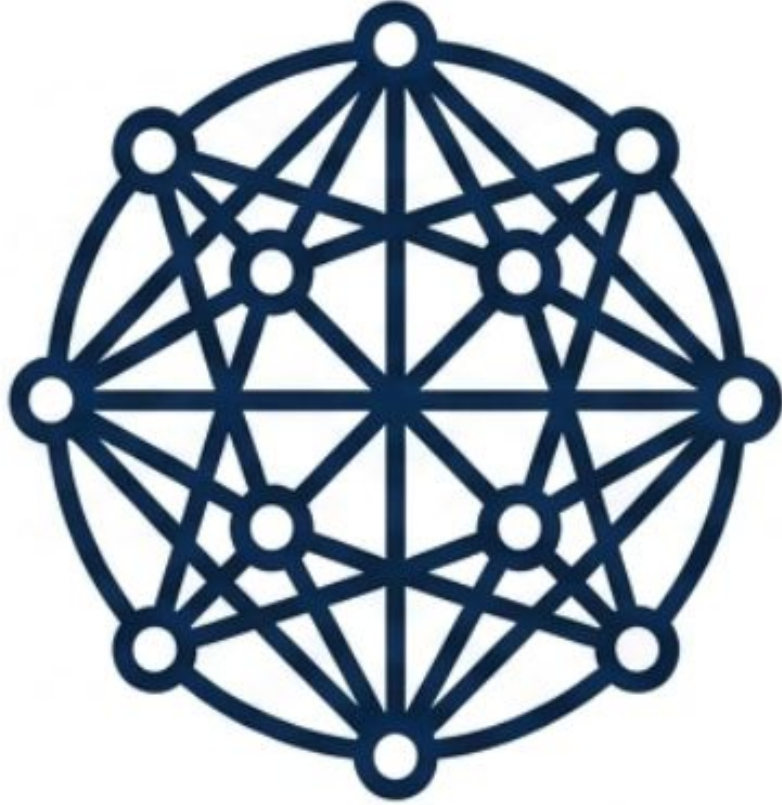


Deterministic Alternatives

- Scatter Search
- Path Relinking

These methods forego heavy randomness, finding new solutions by generating convex or concave linear combinations of real-valued vectors representing high-quality solutions.

Key Takeaways: The Search for the Optimum



1. The NP-Hard Reality

Combinatorial problems demand heuristic shortcuts; complete exhaustive search is mathematically impossible at scale due to exponential time demands.



2. The Local Trap

Deterministic approaches (Greedy, Local Search) blindly climb toward the nearest peak, fatally locking themselves in suboptimal local states.



3. Structured Randomness

Metaheuristics like Simulated Annealing leverage thermodynamic principles, probabilistically accepting worse solutions to navigate out of traps and discover the true global optimum.

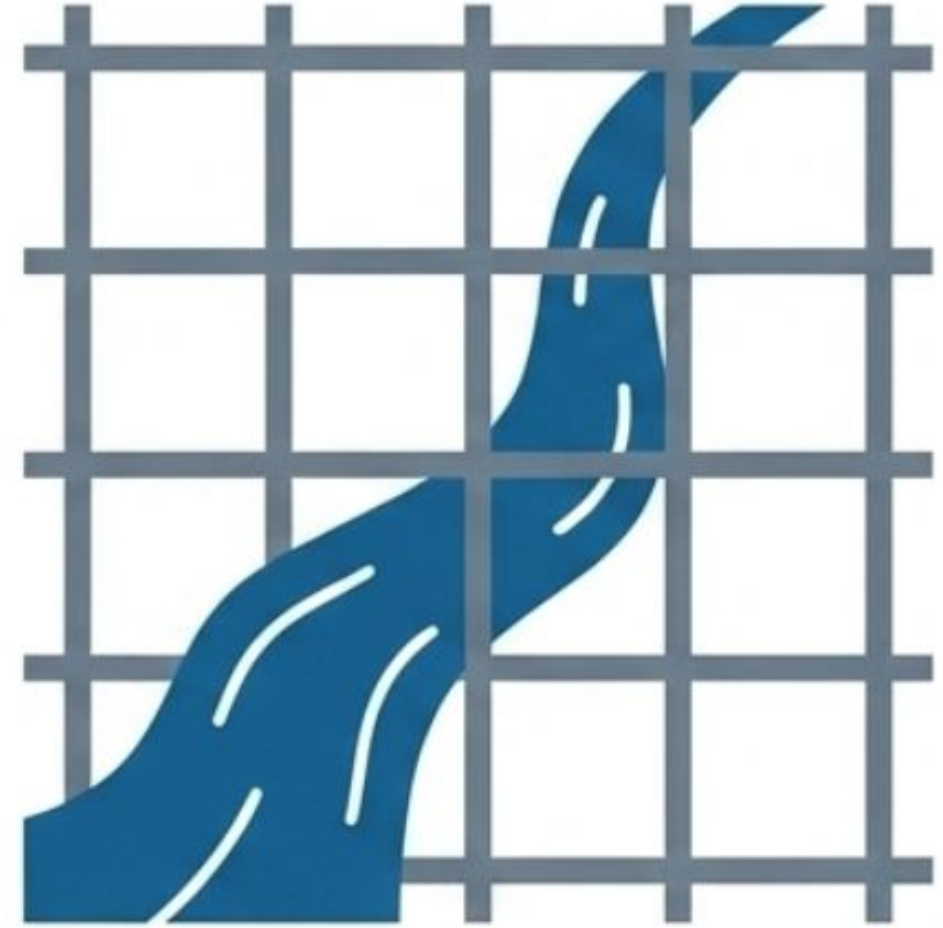
Synthesis: Exact Integrations & Hybrid Frameworks

Memetic Algorithms



Marrying population-based recombination with local search to refine offspring.

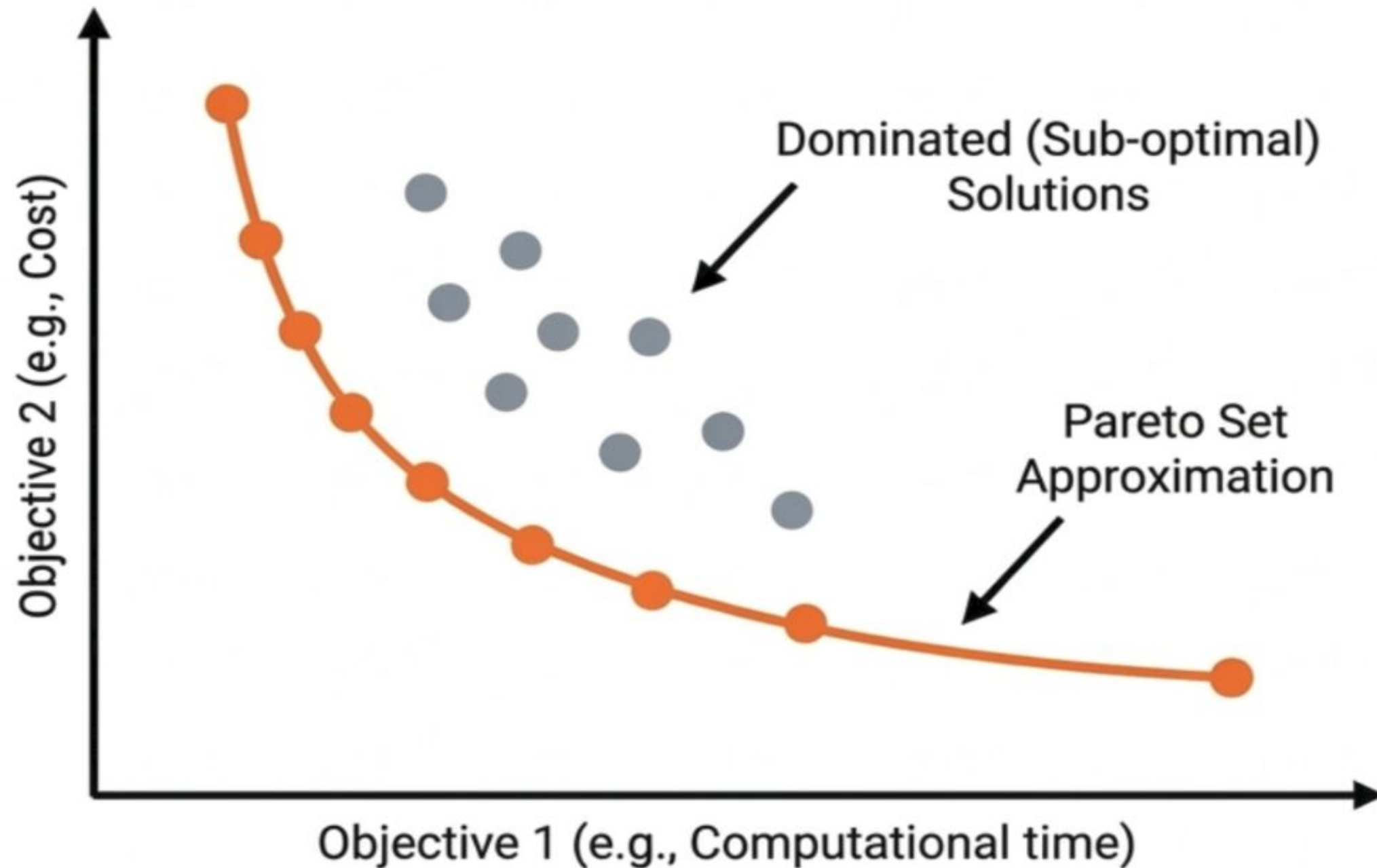
Matheuristics



Integrating exact procedures to solve complex sub-problems generated by metaheuristic decomposition (e.g., RINS).

Hybridization: The era of isolated algorithms is over. Modern implementations layer a Constructive phase for initial solutions, Local Search for refinement, and Exact solvers to optimize sub-components.

Multi-Objective Optimization: The Pareto Front



In multi-objective problems, a single optimum is meaningless. Algorithms naturally utilize populations to approximate the Pareto Front—a set of mutually non-dominated solutions representing optimal trade-offs.

Stochastic Optimization: The Reality of Uncertainty



Closed-Form Expression

Evaluating exact expected values mathematically (if available).

Statistical Approximation

Estimating bounds using statistical models.

Monte Carlo Simulation

Running repeated random sampling to ascertain empirical probabilities.

Real-world systems are rarely static. Stochastic metaheuristics embrace uncertainty, optimizing for expected values rather than deterministic illusions.

Lab / Homework Activities

1. Based on the references below, prepare a **3-page tutorial article** in PDF with a greedy heuristic for the traveling salesman problem (TSP). Provide computational times and a graphical representation of your solutions. Include Python / Julia code in a separate file.

References:

- Jünger, M., Reinelt, G., & Rinaldi, G. (1995). The traveling salesman problem. Handbooks in operations research and management science, 7, 225-330.
- Alanzi, E., & Menai, M. E. B. (2025). Solving the traveling salesman problem with machine learning: a review of recent advances and challenges. Artificial Intelligence Review, 58(9), 267.
- <https://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/>

